

254 类的基本思想是数据抽象 (data abstraction) 和封装 (encapsulation)。数据抽象是一种依赖于接口 (interface) 和实现 (implementation) 分离的编程 (以及设计) 技术。类的接口包括用户所能执行的操作; 类的实现则包括类的数据成员、负责接口实现的函数体以及定义类所需的各种私有函数。

封装实现了类的接口和实现的分离。封装后的类隐藏了它的实现细节, 也就是说, 类的用户只能使用接口而无法访问实现部分。

类要想实现数据抽象和封装, 需要首先定义一个抽象数据类型 (abstract data type)。在抽象数据类型中, 由类的设计者负责考虑类的实现过程; 使用该类的程序员则只需要抽象地思考类型做了什么, 而无须了解类型的工作细节。

## 7.1 定义抽象数据类型

在第 1 章中使用的 `Sales_item` 类是一个抽象数据类型, 我们通过它的接口 (例如 1.5.1 节 (第 17 页) 描述的操作) 来使用一个 `Sales_item` 对象。我们不能访问 `Sales_item` 对象的数据成员, 事实上, 我们甚至根本不知道这个类有哪些数据成员。

与之相反, `Sales_data` 类 (参见 2.6.1 节, 第 64 页) 不是一个抽象数据类型。它允许类的用户直接访问它的数据成员, 并且要求由用户来编写操作。要想把 `Sales_data` 变成抽象数据类型, 我们需要定义一些操作以供类的用户使用。一旦 `Sales_data` 定义了自己自己的操作, 我们就可以封装 (隐藏) 它的数据成员了。



### 7.1.1 设计 `Sales_data` 类

我们的最终目的是令 `Sales_data` 支持与 `Sales_item` 类完全一样的操作集合。`Sales_item` 类有一个名为 `isbn` 的成员函数 (member function) (参见 1.5.2 节, 第 20 页), 并且支持 `+`、`=`、`+=`、`<<` 和 `>>` 运算符。

我们将在第 14 章学习如何自定义运算符。现在, 我们先为这些运算定义普通 (命名的) 函数形式。由于 14.1 节 (第 490 页) 将要解释的原因, 执行加法和 IO 的函数不作为 `Sales_data` 的成员, 相反的, 我们将其定义成普通函数; 执行复合赋值运算的函数是成员函数。`Sales_data` 类无须专门定义赋值运算, 其原因将在 7.1.5 节 (第 239 页) 介绍。

综上所述, `Sales_data` 的接口应该包含以下操作:

- 一个 `isbn` 成员函数, 用于返回对象的 ISBN 编号
- 一个 `combine` 成员函数, 用于将一个 `Sales_data` 对象加到另一个对象上
- 一个名为 `add` 的函数, 执行两个 `Sales_data` 对象的加法
- 一个 `read` 函数, 将数据从 `istream` 读入到 `Sales_data` 对象中
- 一个 `print` 函数, 将 `Sales_data` 对象的值输出到 `ostream`

255

### 关键概念: 不同的编程角色

程序员们常把运行其程序的人称作用户 (user)。类似的, 类的设计者也是为其用户设计并实现一个类的人; 显然, 类的用户是程序员, 而非应用程序的最终使用者。

当我们提及“用户”一词时, 不同的语境决定了不同的含义。如果我们说用户代码或者 `Sales_data` 类的用户, 指的是使用类的程序员; 如果我们说书店应用程序的用

户，则意指运行该应用程序的书店经理。



C++程序员们无须刻意区分应用程序的用户以及类的用户。

在一些简单的应用程序中，类的用户和类的设计者常常是同一个人。尽管如此，还是最好把角色区分开来。当我们设计类的接口时，应该考虑如何才能使得类易于使用；而当我们使用类时，不应该顾及类的实现机理。

要想开发一款成功的应用程序，其作者必须充分了解并实现用户的需求。同样，优秀的类设计者也应该密切关注那些有可能使用该类的程序员的需求。作为一个设计良好的类，既要有直观且易于使用的接口，也必须具备高效的实现过程。

### 使用改进的 Sales\_data 类

在考虑如何实现我们的类之前，首先来看看应该如何使用上面这些接口函数。举个例子，我们使用这些函数编写 1.6 节（第 21 页）书店程序的另外一个版本，其中不再使用 Sales\_item 对象，而是使用 Sales\_data 对象：

```

Sales_data total;                // 保存当前求和结果的变量
if (read(cin, total)) {         // 读入第一笔交易
    Sales_data trans;           // 保存下一条交易数据的变量
    while(read(cin, trans)) {   // 读入剩余的交易
        if (total.isbn() == trans.isbn()) // 检查 isbn
            total.combine(trans); // 更新变量 total 当前的值
        else {
            print(cout, total) << endl; // 输出结果
            total = trans;             // 处理下一本书
        }
    }
    print(cout, total) << endl; // 输出最后一条交易
} else {                         // 没有输入任何信息
    cerr << "No data?!" << endl; // 通知用户
}

```

一开始我们定义了一个 Sales\_data 对象用于保存实时的汇总信息。在 if 条件内部，调用 read 函数将第一条交易读入到 total 中，这里的条件部分与之前我们使用 >> 运算符的效果是一样的。read 函数返回它的流参数，而条件部分负责检查这个返回值（参见 4.11.2 节，第 144 页），如果 read 函数失败，程序将直接跳转到 else 语句并输出一条错误信息。

◀ 256

如果检测到读入了数据，我们定义变量 trans 用于存放每一条交易。while 语句的条件部分同样是检查 read 函数的返回值，只要输入操作成功，条件就被满足，意味着我们可以处理一条新的交易。

在 while 循环内部，我们分别调用 total 和 trans 的 isbn 成员以比较它们的 ISBN 编号。如果 total 和 trans 指示的是同一本书，我们调用 combine 函数将 trans 的内容添加到 total 表示的实时汇总结果中去。如果 trans 指示的是一本新书，我们调用 print 函数将之前一本书的汇总信息输出出来。因为 print 返回的是它的流参数的引用，所以我们可以把 print 的返回值作为 << 运算符的左侧运算对象。通过这种方式，我们输出 print 函数的处理结果，然后转到下一行。接下来，把 trans 赋给 total，从而为接着处理文件中下一本书的记录做好了准备。

处理完所有输入数据后,使用 while 循环之后的 print 语句将最后一条交易的信息输出出来。

### 7.1.1 节练习

**练习 7.1:** 使用 2.6.1 节练习定义的 Sales\_data 类为 1.6 节 (第 21 页) 的交易处理程序编写一个新版本。



### 7.1.2 定义改进的 Sales\_data 类

改进之后的类的数据成员将与 2.6.1 节 (第 64 页) 定义的版本保持一致,它们包括: bookNo, string 类型,表示 ISBN 编号; units\_sold, unsigned 类型,表示某本书的销量;以及 revenue, double 类型,表示这本书的总销售收入。

如前所述,我们的类将包含两个成员函数: combine 和 isbn。此外,我们还将赋予 Sales\_data 另一个成员函数用于返回售出书籍的平均价格,这个函数被命名为 avg\_price。因为 avg\_price 的目的并非通用,所以它应该属于类的实现的一部分,而非接口的一部分。

定义 (参见 6.1 节,第 182 页) 和声明 (参见 6.1.2 节,第 186 页) 成员函数的方式与普通函数差不多。成员函数的声明必须在类的内部,它的定义则既可以在类的内部也可以在类的外部。作为接口组成部分的非成员函数,例如 add、read 和 print 等,它们的定义和声明都在类的外部。

由此可知,改进的 Sales\_data 类应该如下所示:

```
struct Sales_data {
    // 新成员: 关于 Sales_data 对象的操作
    std::string isbn() const { return bookNo; }
    Sales_data& combine(const Sales_data&);
    double avg_price() const;
    // 数据成员和 2.6.1 节 (第 64 页) 相比没有改变
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
// Sales_data 的非成员接口函数
Sales_data add(const Sales_data&, const Sales_data&);
std::ostream &print(std::ostream&, const Sales_data&);
std::istream &read(std::istream&, Sales_data&);
```

257

Note

定义在类内部的函数是隐式的 inline 函数 (参见 6.5.2 节,第 214 页)。

### 定义成员函数

尽管所有成员都必须在类的内部声明,但是成员函数体可以定义在类内也可以定义在类外。对于 Sales\_data 类来说, isbn 函数定义在了类内,而 combine 和 avg\_price 定义在了类外。

我们首先介绍 isbn 函数,它的参数列表为空,返回值是一个 string 对象:

```
std::string isbn() const { return bookNo; }
```

和其他函数一样，成员函数体也是一个块。在此例中，块只有一条 `return` 语句，用于返回 `Sales_data` 对象的 `bookNo` 数据成员。关于 `isbn` 函数一件有意思的事情是：它是如何获得 `bookNo` 成员所依赖的对象的呢？

## 引入 `this`

让我们再一次观察对 `isbn` 成员函数的调用：

```
total.isbn()
```

在这里，我们使用了点运算符（参见 4.6 节，第 133 页）来访问 `total` 对象的 `isbn` 成员，然后调用它。

7.6 节（第 268 页）将介绍一种例外的形式，当我们调用成员函数时，实际上是在替某个对象调用它。如果 `isbn` 指向 `Sales_data` 的成员（例如 `bookNo`），则它隐式地指向调用该函数的对象的成员。在上面所示的调用中，当 `isbn` 返回 `bookNo` 时，实际上它隐式地返回 `total.bookNo`。

成员函数通过一个名为 `this` 的额外的隐式参数来访问调用它的那个对象。当我们调用一个成员函数时，用请求该函数的对象地址初始化 `this`。例如，如果调用

```
total.isbn()
```

则编译器负责把 `total` 的地址传递给 `isbn` 的隐式形参 `this`，可以等价地认为编译器将该调用重写成了如下的形式：

```
// 伪代码，用于说明调用成员函数的实际执行过程
Sales_data::isbn(&total)
```



成员函数的 `this` 形参实际上是 `const T* const this`

◀ 258

其中，调用 `Sales_data` 的 `isbn` 成员时传入了 `total` 的地址。

在成员函数内部，我们可以直接使用调用该函数的对象的成员，而无须通过成员访问运算符来做到这一点，因为 `this` 所指的正是这个对象。任何对类成员的直接访问都被看作 `this` 的隐式引用，也就是说，当 `isbn` 使用 `bookNo` 时，它隐式地使用 `this` 指向的成员，就像我们书写了 `this->bookNo` 一样。

对于我们来说，`this` 形参是隐式定义的。实际上，任何自定义名为 `this` 的参数或变量的行为都是非法的。我们可以在成员函数体内部使用 `this`，因此尽管没有必要，但我们还是能把 `isbn` 定义成如下的形式：

```
std::string isbn() const { return this->bookNo; }
```

因为 `this` 的目的总是指向“这个”对象，所以 `this` 是一个常量指针（参见 2.4.2 节，第 56 页），我们不允许改变 `this` 中保存的地址。

## 引入 `const` 成员函数

`isbn` 函数的另一个关键之处是紧随参数列表之后的 `const` 关键字，这里，`const` 的作用是修改隐式 `this` 指针的类型。

默认情况下，`this` 的类型是指向类类型非常量版本的常量指针。例如在 `Sales_data` 成员函数中，`this` 的类型是 `Sales_data *const`。尽管 `this` 是隐式的，但它仍然需要遵循初始化规则，意味着（在默认情况下）我们不能把 `this` 绑定到一个常量对象上（参见 2.4.2 节，第 56 页）。这一情况也就使得我们不能在一个常量对象上调用普通的成员函数。

如果 `isbn` 是一个普通函数而且 `this` 是一个普通的指针参数, 则我们应该把 `this` 声明成 `const Sales_data *const`。毕竟, 在 `isbn` 的函数体内不会改变 `this` 所指的對象, 所以把 `this` 设置为指向常量的指针有助于提高函数的灵活性。

然而, `this` 是隐式的并且不会出现在参数列表中, 所以在哪儿将 `this` 声明成指向常量的指针就成为我们必须面对的问题。C++ 语言的做法是允许把 `const` 关键字放在成员函数的参数列表之后, 此时, 紧跟在参数列表后面的 `const` 表示 `this` 是一个指向常量的指针。像这样使用 `const` 的成员函数被称作**常量成员函数** (const member function)。

可以把 `isbn` 的函数体想象成如下的形式:

```
// 伪代码, 说明隐式的 this 指针是如何使用的
// 下面的代码是非法的: 因为我们不能显式地定义自己的 this 指针
// 谨记此处的 this 是一个指向常量的指针, 因为 isbn 是一个常量成员
std::string Sales_data::isbn(const Sales_data *const this)
{ return this->isbn; }
```

因为 `this` 是指向常量的指针, 所以常量成员函数不能改变调用它的对象的内容。在上例中, `isbn` 可以读取调用它的对象的数据成员, 但是不能写入新值。

259



常量对象, 以及常量对象的引用或指针都只能调用常量成员函数。

## 类作用域和成员函数

回忆之前我们所学的知识, 类本身就是一个作用域 (参见 2.6.1 节, 第 64 页)。类的成员函数的定义嵌套在类的作用域之内, 因此, `isbn` 中用到的名字 `bookNo` 其实就是定义在 `Sales_data` 内的数据成员。

值得注意的是, 即使 `bookNo` 定义在 `isbn` 之后, `isbn` 也还是能够使用 `bookNo`。就如我们将在 7.4.1 节 (第 254 页) 学习到的那样, 编译器分两步处理类: 首先编译成员的声明, 然后才轮到成员函数体 (如果有的话)。因此, 成员函数体可以随意使用类中的其他成员而无须在意这些成员出现的次序。

## 在类的外部定义成员函数

像其他函数一样, 当我们在类的外部定义成员函数时, 成员函数的定义必须与它的声明匹配。也就是说, 返回类型、参数列表和函数名都得与类内部的声明保持一致。如果成员被声明成常量成员函数, 那么它的定义也必须在参数列表后明确指定 `const` 属性。同时, 类外部定义的成员的名字必须包含它所属的类名:

```
double Sales_data::avg_price() const {
    if (units_sold)
        return revenue/units_sold;
    else
        return 0;
}
```

函数名 `Sales_data::avg_price` 使用作用域运算符 (参见 1.2 节, 第 7 页) 来说明如下的事实: 我们定义了一个名为 `avg_price` 的函数, 并且该函数被声明在类 `Sales_data` 的作用域内。一旦编译器看到这个函数名, 就能理解剩余的代码是位于类的作用域内的。因此, 当 `avg_price` 使用 `revenue` 和 `units_sold` 时, 实际上它隐式地使用了



Sales\_data 的成员。

### 定义一个返回 this 对象的函数

函数 combine 的设计初衷类似于复合赋值运算符+=，调用该函数的对象代表的是赋值运算符左侧的运算对象，右侧运算对象则通过显式的实参传入函数：

```
Sales_data& Sales_data::combine(const Sales_data &rhs)
{
    units_sold += rhs.units_sold; // 把 rhs 的成员加到 this 对象的成员上
    revenue += rhs.revenue;
    return *this;                // 返回调用该函数的对象
}
```

当我们的交易处理程序调用如下的函数时，

```
total.combine(trans);           // 更新变量 total 当前的值
```

total 的地址被绑定到隐式的 this 参数上，而 rhs 绑定到了 trans 上。因此，当 combine 执行下面的语句时，

```
units_sold += rhs.units_sold;   // 把 rhs 的成员添加到 this 对象的成员中
```

效果等同于求 total.units\_sold 和 trans.unit\_sold 的和，然后把结果保存到 total.units\_sold 中。

该函数一个值得关注的部分是它的返回类型和返回语句。一般来说，当我们定义的函数类似于某个内置运算符时，应该令该函数的行为尽量模仿这个运算符。内置的赋值运算符把它的左侧运算对象当成左值返回（参见 4.4 节，第 129 页），因此为了与它保持一致，combine 函数必须返回引用类型（参见 6.3.2 节，第 202 页）。因为此时的左侧运算对象是一个 Sales\_data 的对象，所以返回类型应该是 Sales\_data&。

如前所述，我们无须使用隐式的 this 指针访问函数调用者的某个具体成员，而是需要把调用函数的对象当成一个整体来访问：

```
return *this;                   // 返回调用该函数的对象
```

其中，return 语句解引用 this 指针以获得执行该函数的对象，换句话说，上面的这个调用返回 total 的引用。

## 7.1.2 节练习

**练习 7.2：**曾在 2.6.2 节的练习（第 67 页）中编写了一个 Sales\_data 类，请向这个类添加 combine 和 isbn 成员。

**练习 7.3：**修改 7.1.1 节（第 229 页）的交易处理程序，令其使用这些成员。

**练习 7.4：**编写一个名为 Person 的类，使其表示人员的姓名和住址。使用 string 对象存放这些元素，接下来的练习将不断充实这个类的其他特征。

**练习 7.5：**在你的 Person 类中提供一些操作使其能够返回姓名和住址。这些函数是否应该是 const 的呢？解释原因。



### 7.1.3 定义类相关的非成员函数

类的作者常常需要定义一些辅助函数，比如 `add`、`read` 和 `print` 等。尽管这些函数定义的操作从概念上来说属于类的接口的组成部分，但它们实际上并不属于类本身。

我们定义非成员函数的方式与定义其他函数一样，通常把函数的声明和定义分离开来（参见 6.1.2 节，第 168 页）。如果函数在概念上属于类但是不定义在类中，则它一般应与类声明（而非定义）在同一个头文件内。在这种方式下，用户使用接口的任何部分都只需要引入一个文件。



一般来说，如果非成员函数是类接口的组成部分，则这些函数的声明应该与类在同一个头文件内。

#### 定义 `read` 和 `print` 函数

下面的 `read` 和 `print` 函数与 2.6.2 节（第 66 页）中的代码作用一样，而且代码本身也非常相似：

```
// 输入的交易信息包括 ISBN、售出总数和售出价格
istream &read(istream &is, Sales_data &item)
{
    double price = 0;
    is >> item.bookNo >> item.units_sold >> price;
    item.revenue = price * item.units_sold;
    return is;
}

ostream &print(ostream &os, const Sales_data &item)
{
    os << item.isbn() << " " << item.units_sold << " "
        << item.revenue << " " << item.avg_price();
    return os;
}
```

`read` 函数从给定流中将数据读到给定的对象里，`print` 函数则负责将给定对象的内容打印到给定的流中。

除此之外，关于上面的函数还有两点是非常重要的。第一点，`read` 和 `print` 分别接受一个各自 IO 类型的引用作为其参数，这是因为 IO 类属于不能被拷贝的类型，因此我们只能通过引用来传递它们（参见 6.2.2 节，第 188 页）。而且，因为读取和写入的操作会改变流的内容，所以两个函数接受的都是普通引用，而非对常量的引用。

第二点，`print` 函数不负责换行。一般来说，执行输出任务的函数应该尽量减少对格式的控制，这样可以确保由用户代码来决定是否换行。

#### 定义 `add` 函数

`add` 函数接受两个 `Sales_data` 对象作为其参数，返回值是一个新的 `Sales_data`，用于表示前两个对象的和：

```
Sales_data add(const Sales_data &lhs, const Sales_data &rhs)
{
    Sales_data sum = lhs;           // 把 lhs 的数据成员拷贝给 sum
```

```

    sum.combine(rhs);           // 把 rhs 的数据成员加到 sum 当中
    return sum;
}

```

在函数体中，我们定义了一个新的 `Sales_data` 对象并将其命名为 `sum`。`sum` 将用于存放两笔交易的和，我们用 `lhs` 的副本来初始化 `sum`。默认情况下，拷贝类的对象其实拷贝的是对象的数据成员。在拷贝工作完成之后，`sum` 的 `bookNo`、`units_sold` 和 `revenue` 将与 `lhs` 一致。接下来我们调用 `combine` 函数，将 `rhs` 的 `units_sold` 和 `revenue` 添加给 `sum`。最后，函数返回 `sum` 的副本。

262

### 7.1.3 节练习

**练习 7.6:** 对于函数 `add`、`read` 和 `print`，定义你自己的版本。

**练习 7.7:** 使用这些新函数重写 7.1.2 节（第 233 页）练习中的交易处理程序。

**练习 7.8:** 为什么 `read` 函数将其 `Sales_data` 参数定义成普通的引用，而 `print` 将其参数定义成常量引用？

**练习 7.9:** 对于 7.1.2 节（第 233 页）练习中的代码，添加读取和打印 `Person` 对象的操作。

**练习 7.10:** 在下面这条 `if` 语句中，条件部分的作用是什么？

```
if (read(read(cin, data1), data2))
```

### 7.1.4 构造函数

每个类都分别定义了它的对象被初始化的方式，类通过一个或几个特殊的成员函数来控制其对象的初始化过程，这些函数叫做**构造函数**（**constructor**）。构造函数的任务是初始化类对象的数据成员，无论何时只要类的对象被创建，就会执行构造函数。

在这一节中，我们将介绍定义构造函数的基础知识。构造函数是一个非常复杂的问题，我们还会在 7.5 节（第 257 页）、15.7 节（第 551 页）、18.1.3 节（第 689 页）和第 13 章介绍更多关于构造函数的知识。

构造函数的名字和类名相同。和其他函数不一样的是，构造函数没有返回类型；除此之外类似于其他的函数，构造函数也有一个（可能为空的）参数列表和一个（可能为空的）函数体。类可以包含多个构造函数，和其他重载函数差不多（参见 6.4 节，第 206 页），不同的构造函数之间必须在参数数量或参数类型上有所区别。

不同于其他成员函数，构造函数不能被声明成 `const` 的（参见 7.1.2 节，第 231 页）。当我们创建类的一个 `const` 对象时，直到构造函数完成初始化过程，对象才能真正取得其“常量”属性。因此，构造函数在 `const` 对象的构造过程中可以向其写值。

#### 合成的默认构造函数

我们的 `Sales_data` 类并没有定义任何构造函数，可是之前使用了 `Sales_data` 对象的程序仍然可以正确地编译和运行。举个例子，第 229 页的程序定义了两个对象：

```

Sales_data total;           // 保存当前求和结果的变量
Sales_data trans;          // 保存下一条交易数据的变量

```





263 这时我们不禁要问: `total` 和 `trans` 是如何初始化的呢?

我们没有为这些对象提供初始值, 因此我们知道它们执行了默认初始化 (参见 2.2.1 节, 第 40 页)。类通过一个特殊的构造函数来控制默认初始化过程, 这个函数叫做**默认构造函数** (default constructor)。默认构造函数无须任何实参。

如我们所见, 默认构造函数在很多方面都有其特殊性。其中之一是, 如果我们的类没有显式地定义构造函数, 那么编译器就会为我们隐式地定义一个默认构造函数。

编译器创建的构造函数又被称为**合成的默认构造函数** (synthesized default constructor)。对于大多数类来说, 这个合成的默认构造函数将按照如下规则初始化类的数据成员:

- 如果存在类内的初始值 (参见 2.6.1 节, 第 64 页), 用它来初始化成员。
- 否则, 默认初始化 (参见 2.2.1 节, 第 40 页) 该成员。

因为 `Sales_data` 为 `units_sold` 和 `revenue` 提供了初始值, 所以合成的默认构造函数将使用这些值来初始化对应的成员; 同时, 它把 `bookNo` 默认初始化成一个空字符串。

### 某些类不能依赖于合成的默认构造函数

合成的默认构造函数只适合非常简单的类, 比如现在定义的这个 `Sales_data` 版本。对于一个普通的类来说, 必须定义它自己的默认构造函数, 原因有三: 第一个原因也是最容易理解的一个原因就是编译器只有在发现类不包含任何构造函数的情况下才会替我们生成一个默认的构造函数。一旦我们定义了一些其他的构造函数, 那么除非我们再定义一个默认的构造函数, 否则类将没有默认构造函数。这条规则的依据是, 如果一个类在某种情况下需要控制对象初始化, 那么该类很可能在所有情况下都需要控制。



只有当类没有声明任何构造函数时, 编译器才会自动地生成默认构造函数。

第二个原因是对于某些类来说, 合成的默认构造函数可能执行错误的操作。回忆我们之前介绍过的, 如果定义在块中的内置类型或复合类型 (比如数组和指针) 的对象被默认初始化 (参见 2.2.1 节, 第 40 页), 则它们的值将是未定义的。该准则同样适用于默认初始化的内置类型成员。因此, 含有内置类型或复合类型成员类应该在类的内部初始化这些成员, 或者定义一个自己的默认构造函数。否则, 用户在创建类的对象时就可能得到未定义的值。



如果类包含有内置类型或者复合类型的成员, 则只有当这些成员全都被赋予了类内的初始值时, 这个类才适合于使用合成的默认构造函数。

264 第三个原因是有的时候编译器不能为某些类合成默认的构造函数。例如, 如果类中包含一个其他类类型的成员且这个成员的类型没有默认构造函数, 那么编译器将无法初始化该成员。对于这样的类来说, 我们必须自定义默认构造函数, 否则该类将没有可用的默认构造函数。在 13.1.6 节 (第 449 页) 中我们将看到还有其他一些情况也会导致编译器无法生成一个正确的默认构造函数。

### 定义 `Sales_data` 的构造函数

对于我们的 `Sales_data` 类来说, 我们将使用下面的参数定义 4 个不同的构造函数:

- 一个 `istream&`, 从中读取一条交易信息。

- 一个 `const string&`, 表示 ISBN 编号; 一个 `unsigned`, 表示售出的图书数量; 以及一个 `double`, 表示图书的售出价格。
- 一个 `const string&`, 表示 ISBN 编号; 编译器将赋予其他成员默认值。
- 一个空参数列表 (即默认构造函数), 正如刚刚介绍的, 既然我们已经定义了其他构造函数, 那么也必须定义一个默认构造函数。

给类添加了这些成员之后, 将得到

```
struct Sales_data {
    // 新增的构造函数
    Sales_data() = default;
    Sales_data(const std::string &s): bookNo(s) { }
    Sales_data(const std::string &s, unsigned n, double p):
        bookNo(s), units_sold(n), revenue(p*n) { }
    Sales_data(std::istream &);
    // 之前已有的其他成员
    std::string isbn() const { return bookNo; }
    Sales_data& combine(const Sales_data&);
    double avg_price() const;
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
```

## = default 的含义

我们从解释默认构造函数的含义开始:

```
Sales_data() = default;
```

首先请明确一点: 因为该构造函数不接受任何实参, 所以它是一个默认构造函数。我们定义这个构造函数的目的仅仅是因为我们既需要其他形式的构造函数, 也需要默认的构造函数。我们希望这个函数的作用完全等同于之前使用的合成默认构造函数。

在 C++11 新标准中, 如果我们需要默认的行为, 那么可以通过在参数列表后面写上 **= default** 来要求编译器生成构造函数。其中, **= default** 既可以和声明一起出现在类的内部, 也可以作为定义出现在类的外部。和其他函数一样, 如果 **= default** 在类的内部, 则默认构造函数是内联的; 如果它在类的外部, 则该成员默认情况下不是内联的。

265

C++  
11



WARNING

上面的默认构造函数之所以对 `Sales_data` 有效, 是因为我们为内置类型的数据成员提供了初始值。如果你的编译器不支持类内初始值, 那么你的默认构造函数就应该使用构造函数初始值列表 (马上就会介绍) 来初始化类的每个成员。

## 构造函数初始值列表

接下来我们介绍类中定义的另外两个构造函数:

```
Sales_data(const std::string &s): bookNo(s) { }
Sales_data(const std::string &s, unsigned n, double p):
    bookNo(s), units_sold(n), revenue(p*n) { }
```

这两个定义中出现了新的部分, 即冒号以及冒号和花括号之间的代码, 其中花括号定义了

(空的)函数体。我们把新出现的部分称为**构造函数初始值列表**(constructor initialize list),它负责为新创建的对象的一个或几个数据成员赋初值。构造函数初始值是成员名字的一个列表,每个名字后面紧跟括号括起来的(或者在花括号内的)成员初始值。不同成员的初始化通过逗号分隔开来。

含有三个参数的构造函数分别使用它的前两个参数初始化成员 `bookNo` 和 `units_sold`, `revenue` 的初始值则通过将售出图书总数和每本书单价相乘计算得到。

只有一个 `string` 类型参数的构造函数使用这个 `string` 对象初始化 `bookNo`, 对于 `units_sold` 和 `revenue` 则没有显式地初始化。当某个数据成员被构造函数初始值列表忽略时,它将以与合成默认构造函数相同的方式隐式初始化。在此例中,这样的成员使用类内初始值初始化,因此只接受一个 `string` 参数的构造函数等价于

```
// 与上面定义的那个构造函数效果相同
Sales_data(const std::string &s):
    bookNo(s), units_sold(0), revenue(0){ }
```

通常情况下,构造函数使用类内初始值不失为一种好的选择,因为只要这样的初始值存在我们就能确保为成员赋予了一个正确的值。不过,如果你的编译器不支持类内初始值,则所有构造函数都应该显式地初始化每个内置类型的成员。

Best  
Practices

构造函数不应该轻易覆盖掉类内的初始值,除非新赋的值与原值不同。如果你不能使用类内初始值,则所有构造函数都应该显式地初始化每个内置类型的成员。

266

有一点需要注意,在上面的两个构造函数中函数体都是空的。这是因为这些构造函数的唯一目的就是为数据成员赋初值,一旦没有其他任务需要执行,函数体也就为空了。

## 在类的外部定义构造函数

与其他几个构造函数不同,以 `istream` 为参数的构造函数需要执行一些实际的操作。在它的函数体内,调用了 `read` 函数以给数据成员赋以初值:

```
Sales_data::Sales_data(std::istream &is)
{
    read(is, *this); // read 函数的作用是从 is 中读取一条交易信息然后
                    // 存入 this 对象中
}
```

构造函数没有返回类型,所以上述定义从我们指定的函数名字开始。和其他成员函数一样,当我们在类的外部定义构造函数时,必须指明该构造函数是哪个类的成员。因此, `Sales_data::Sales_data` 的含义是我们定义 `Sales_data` 类的成员,它的名字是 `Sales_data`。又因为该成员的名字和类名相同,所以它是一个构造函数。

这个构造函数没有构造函数初始值列表,或者讲得更准确一点,它的构造函数初始值列表是空的。尽管构造函数初始值列表是空的,但是由于执行了构造函数体,所以对象的成员仍然能被初始化。

没有出现在构造函数初始值列表中的成员将通过相应的类内初始值(如果存在的话)初始化,或者执行默认初始化。对于 `Sales_data` 来说,这意味着一旦函数开始执行,则 `bookNo` 将被初始化成空 `string` 对象,而 `units_sold` 和 `revenue` 将是 0。

为了更好地理解调用函数 `read` 的意义，要特别注意 `read` 的第二个参数是一个 `Sales_data` 对象的引用。在 7.1.2 节（第 232 页）中曾经提到过，使用 `this` 来把对象当成一个整体访问，而非直接访问对象的某个成员。因此在此例中，我们使用 `*this` 将“`this`”对象作为实参传递给 `read` 函数。

### 7.1.4 节练习

**练习 7.11:** 在你的 `Sales_data` 类中添加构造函数，然后编写一段程序令其用到每个构造函数。

**练习 7.12:** 把只接受一个 `istream` 作为参数的构造函数定义移到类的内部。

**练习 7.13:** 使用 `istream` 构造函数重写第 229 页的程序。

**练习 7.14:** 编写一个构造函数，令其用我们提供的类内初始值显式地初始化成员。

**练习 7.15:** 为你的 `Person` 类添加正确的构造函数。

## 7.1.5 拷贝、赋值和析构



除了定义类的对象如何初始化之外，类还需要控制拷贝、赋值和销毁对象时发生的行为。对象在几种情况下会被拷贝，如我们初始化变量以及以值的方式传递或返回一个对象等（参见 6.2.1 节，第 187 页和 6.3.2 节，第 200 页）。当我们使用了赋值运算符（参见 4.4 节，第 129 页）时会发生对象的赋值操作。当对象不再存在时执行销毁的操作，比如一个局部对象会在创建它的块结束时被销毁（参见 6.1.1 节，第 184 页），当 `vector` 对象（或者数组）销毁时存储在其中的对象也会被销毁。

267

如果我们不主动定义这些操作，则编译器将替我们合成它们。一般来说，编译器生成的版本将对对象的每个成员执行拷贝、赋值和销毁操作。例如在 7.1.1 节（第 229 页）的书店程序中，当编译器执行如下赋值语句时，

```
total = trans; // 处理下一本书的信息
```

它的行为与下面的代码相同

```
// Sales_data 的默认赋值操作等价于：
total.bookNo = trans.bookNo;
total.units_sold = trans.units_sold;
total.revenue = trans.revenue;
```

我们将在第 13 章中介绍如何自定义上述操作。

### 某些类不能依赖于合成的版本



尽管编译器能替我们合成拷贝、赋值和销毁的操作，但是必须要清楚的一点是，对于某些类来说合成的版本无法正常工作。特别是，当类需要分配类对象之外的资源时，合成的版本常常会失效。举个例子，第 12 章将介绍 C++ 程序是如何分配和管理动态内存的。而在 13.1.4 节（第 447 页）我们将会看到，管理动态内存的类通常不能依赖于上述操作的合成版本。

不过值得注意的是，很多需要动态内存的类能（而且应该）使用 `vector` 对象或者 `string` 对象管理必要的存储空间。使用 `vector` 或者 `string` 的类能避免分配和释放内存带来的复杂性。

进一步讲, 如果类包含 `vector` 或者 `string` 成员, 则其拷贝、赋值和销毁的合成版本能够正常工作。当我们对含有 `vector` 成员的对象执行拷贝或者赋值操作时, `vector` 类会设法拷贝或者赋值成员中的元素。当这样的对象被销毁时, 将销毁 `vector` 对象, 也就是依次销毁 `vector` 中的每一个元素。这一点与 `string` 是非常类似的。



在学习第 13 章关于如何自定义操作的知识之前, 类中所有分配的资源都应该直接以类的数据成员的形式存储。



## 7.2 访问控制与封装

268

到目前为止, 我们已经为类定义了接口, 但并没有任何机制强制用户使用这些接口。我们的类还没有封装, 也就是说, 用户可以直达 `Sales_data` 对象的内部并且控制它的具体实现细节。在 C++ 语言中, 我们使用访问说明符 (access specifiers) 加强类的封装性:

- 定义在 **public** 说明符之后的成员在整个程序内可被访问, `public` 成员定义类的接口。
- 定义在 **private** 说明符之后的成员可以被类的成员函数访问, 但是不能被使用该类的代码访问, `private` 部分封装了 (即隐藏了) 类的实现细节。

再一次定义 `Sales_data` 类, 其新形式如下所示:

```
class Sales_data {
public:           // 添加了访问说明符
    Sales_data() = default;
    Sales_data(const std::string &s, unsigned n, double p):
        bookNo(s), units_sold(n), revenue(p*n) { }
    Sales_data(const std::string &s): bookNo(s) { }
    Sales_data(std::istream&);
    std::string isbn() const { return bookNo; }
    Sales_data &combine(const Sales_data&);
private:       // 添加了访问说明符
    double avg_price() const
        { return units_sold ? revenue/units_sold : 0; }
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
```

作为接口的一部分, 构造函数和部分成员函数 (即 `isbn` 和 `combine`) 紧跟在 `public` 说明符之后; 而数据成员和作为实现部分的函数则跟在 `private` 说明符后面。

一个类可以包含 0 个或多个访问说明符, 而且对于某个访问说明符能出现多少次也没有严格限定。每个访问说明符指定了接下来的成员的访问级别, 其有效范围直到出现下一个访问说明符或者到达类的结尾处为止。

### 使用 `class` 或 `struct` 关键字

在上面的定义中我们还做了一个微妙的变化, 我们使用了 **class** 关键字而非 **struct** 开始类的定义。这种变化仅仅是形式上有所不同, 实际上我们可以使用这两个关键字中的任何一个定义类。唯一的一点区别是, `struct` 和 `class` 的默认访问权限不太一样。

类可以在它的第一个访问说明符之前定义成员, 对这种成员的访问权限依赖于类定义